

CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

STACK & QUEUE



TS. TRẦN NGỌC VIỆT
TH.S LÊ CÔNG HIẾU



HỌC KỲ 3 – NĂM HỌC 2022-2023



KHÓA K28 & K27

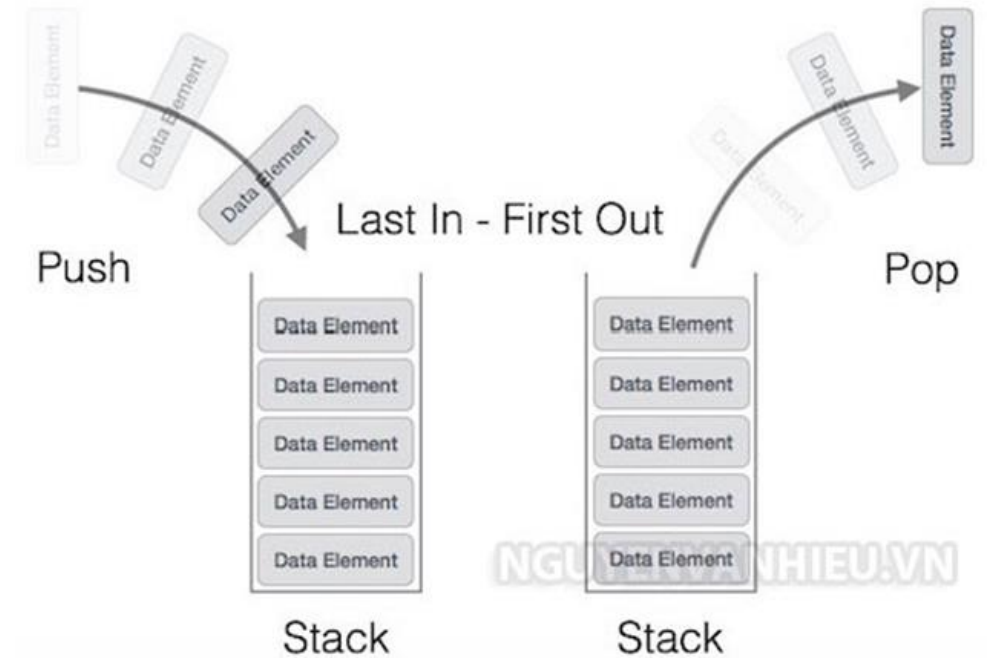
1. Ngăn xếp – stack

-Một ngăn xếp-stack là một cấu trúc dữ liệu hoạt động theo nguyên lý “vào sau ra trước” LIFO- Last In First Out. Tức là, phần tử cuối cùng được chèn vào ngăn xếp sẽ là phần tử đầu tiên được lấy ra khỏi ngăn xếp-stack.

-Chẳng hạn, một chồng sách (minh họa chồng đĩa) và bạn để nó trong một cái hộp như hình phía dưới. Giả sử hộp này vừa khít các cuốn sách. Khi đó, các thao tác:

-Thêm một cuốn sách vào hộp(push của ngăn xếp-stack)

-Lấy một cuốn sách khỏi hộp, bạn chỉ lấy được thằng trên cùng(pop của ngăn xếp-stack)



Hình 1. Ngăn xếp-stack

1. Ngăn xếp – stack

1. PUSH trong cấu trúc dữ liệu ngăn xếp-stack

1.1. Tiến trình các bước đặt thêm phần tử dữ liệu mới vào trên ngăn xếp còn được biết đến với tên chức năng PUSH.

Chức năng **push** bao gồm các bước sau

Bước 1: kiểm tra xem ngăn xếp-stack đã đầy hay chưa.

Bước 2: nếu ngăn xếp-stack là đầy, tiến trình bị lỗi và thoát ra.

Bước 3: nếu ngăn xếp-stack chưa đầy, tăng top để trỏ tới phần bộ nhớ trống tiếp theo.

Bước 4: thêm phần tử dữ liệu vào vị trí nơi mà top đang trỏ đến trên ngăn xếp-stack.

Bước 5: trả về success.

1. Ngăn xếp – stack

1.2. Giải thuật chức năng PUSH của cấu trúc dữ liệu ngăn xếp-stack + giải thuật mã giả như sau

```
bắt đầu chức năng push: stack, data  
    if stack đã đầy  
        return null  
    kết thúc if  
    top  $\leftarrow$  top + 1  
    stack[top]  $\leftarrow$  data  
kết thúc hàm
```

1. Ngăn xếp – stack

2. POP trong cấu trúc dữ liệu ngăn xếp-stack

- Thực hiện chức năng POP xóa phần tử từ ngăn xếp-stack còn được gọi là POP.
- Triển khai mảng của chức năng pop(), phần tử dữ liệu không thực sự bị xóa, thay vào đó **top** sẽ bị giảm về vị trí thấp hơn trong ngăn xếp-stack để trở tới giá trị tiếp theo.
- Danh sách liên kết dữ liệu, pop() xóa phần tử dữ liệu và xóa phần tử khỏi không gian bộ nhớ.

Chức năng POP bao gồm các bước:

Bước 1: kiểm tra ngăn xếp-stack là trống hay không.

Bước 2: nếu ngăn xếp-stack đầy, tiến trình bị lỗi và thoát ra.

Bước 3: nếu ngăn xếp-stack là không trống, truy cập phần tử dữ liệu tại top đang trở tới.

Bước 4: giảm giá trị của top đi 1.

Bước 5: trả về success.

1. Ngăn xếp – stack

+Giải thuật cho chức năng POP bằng mã giả

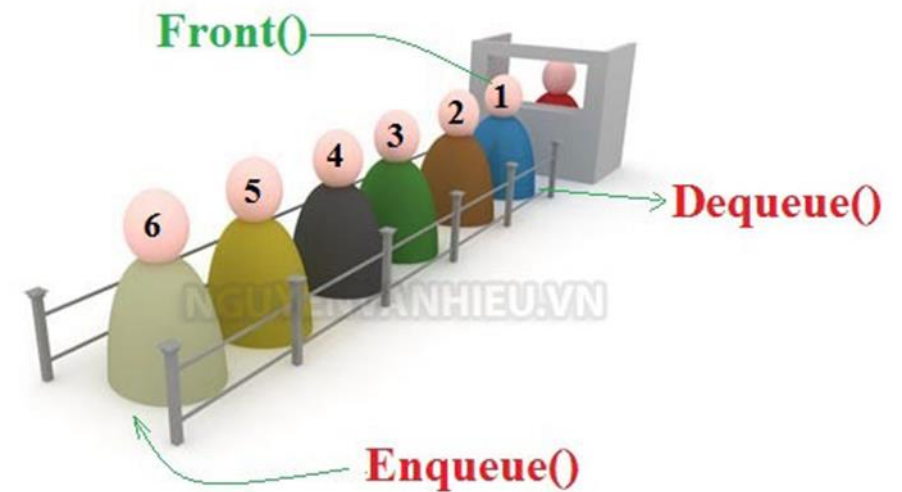
```
bắt đầu hàm pop: stack, data
    if stack là trống
        return null
    kết thúc if
    data ← stack[top]
    top ← top - 1
    return data
kết thúc hàm
```

2. Hàng đợi – queue

-Một hàng đợi-queue là một cấu trúc dữ liệu dùng để lưu trữ các đối tượng theo cơ chế FIFO -First In First Out.

-Sắp xếp hàng đợi-queue rất hay gặp trong đời sống hàng ngày. Chẳng hạn, xếp hàng vào siêu thị dưới đây là một mô phỏng rất dễ hiểu.

-Cấu trúc hàng đợi-queue, có thể thêm phần tử vào một đầu của queue(cuối hàng đợi), và có thể xóa phần tử ở đầu của hàng đợi-queue(đầu hàng đợi).



Hình 2. Hàng đợi-queue

2. Hàng đợi – queue

-Cấu trúc hàng đợi-queue, có các chức năng như sau

- EnQueue: Thêm phần tử vào cuối(rear) của hàng đợi-queue.
- DeQueue: Xóa phần tử khỏi đầu(front) của hàng đợi-queue. Nếu hàng đợi-queue rỗng thì thông báo lỗi.
- IsEmpty: Kiểm tra hàng đợi-queue rỗng.
- Front: Lấy giá trị của phần tử ở đầu(front) của hàng đợi-queue. Lấy giá trị không làm thay đổi hàng đợi-queue.
- *queue[]*: Một mảng một chiều mô phỏng cho hàng đợi
- *arraySize*: Số lượng phần tử tối đa có thể lưu trữ trong *queue[]*
- *front*: Chỉ số của phần tử ở đầu queue. Nó sẽ là chỉ số của phần tử sẽ bị xóa ở lần tiếp theo
- *rear*: Chỉ số của phần tử tiếp theo sẽ được thêm vào cuối của queue

2. Hàng đợi – queue

+Enqueue – Thêm vào cuối hàng đợi

Nếu hàng đợi-queue chưa đầy, có thể thêm phần tử cần thêm vào cuối(*rear*) của hàng đợi.

Ngược lại, thông báo lỗi.

Trường hợp, *rear* là chỉ số của phần tử sẽ được thêm ở lần tiếp theo. Do đó, thêm xong rồi ta mới tăng *rear* lên 1 đơn vị. Giá trị *rear* cần thay đổi nên được truyền theo tham chiếu.

2. Hàng đợi – queue

#Hoạt động Enqueue

Hàng đợi duy trì con trỏ dữ liệu gồm phía trước và phía sau .

Các bước sau, thực hiện để ghi dữ liệu (chèn) vào hàng đợi như sau

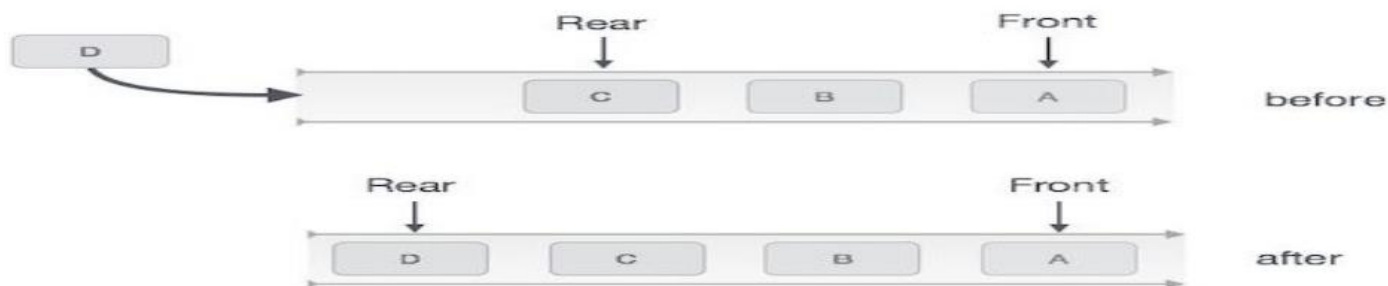
Bước 1 - Kiểm tra hàng đợi đã đầy chưa.

Bước 2 - Nếu hàng đợi đầy, tạo ra lỗi (tràn) và thoát.

Bước 3 - Nếu hàng đợi chưa đầy, tăng con trỏ phía sau để trở đến khoảng trống tiếp theo.

Bước 4 - Thêm phần tử dữ liệu vào vị trí hàng đợi, nơi phía sau đang chỉ.

Bước 5 - Trả về hoàn thành.



Hình – Queue Enqueue

2. Hàng đợi – queue

+ Dequeue – Xóa khỏi đầu hàng đợi

Nếu hàng đợi-queue có ít nhất 1 phần tử, chúng ta sẽ tiến hành xóa bỏ phần tử ở đầu của hàng đợi bằng cách tăng front lên 1 giá trị.

Ở đây, front đang là chỉ số của phần tử sẽ bị xóa rồi. Cho nên chỉ cần tăng là xong, đó cũng là lý do ta cần truyền tham số front sử dụng tham chiếu.

2. Hàng đợi – queue

#Hoạt động Dequeue

Truy cập dữ liệu từ hàng đợi là một quá trình gồm nhiệm vụ - truy cập dữ liệu, trong đó mặt trước đang trỏ và xóa dữ liệu sau khi truy cập.

Các bước được thực hiện để thao tác dequeue như sau

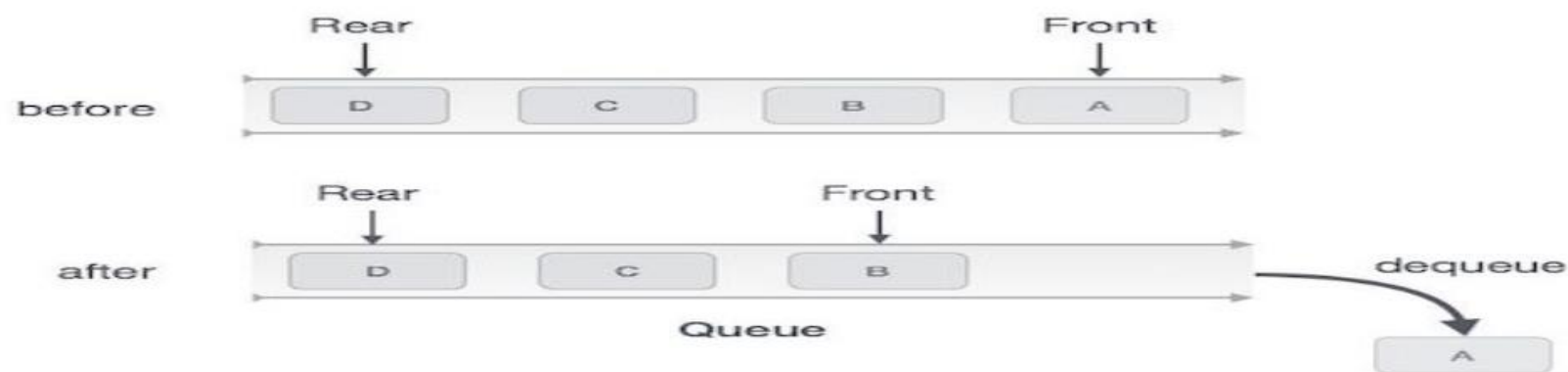
Bước 1 - Kiểm tra xem Hàng đợi có trống không.

Bước 2 - Nếu Hàng đợi trống, tạo ra lỗi (tràn) và thoát.

Bước 3 - Nếu hàng đợi không trống, truy cập dữ liệu nơi mặt trước đang trỏ.

Bước 4 - Tăng con trỏ phía trước để trỏ đến phần tử dữ liệu có sẵn tiếp theo.

Bước 5 - Trả lại hoàn thành.



Hình – Queue Dequeue

#Thực hành 1.1:

```
myStack = []          #??  
  
myStack.append('data science')    #??  
myStack.append('data analytics')  
myStack.append('data structures and algorithms')  
myStack.append('big data')  
myStack.append('learning data analytics')  
myStack  
  
myStack.pop()         #??  
myStack.pop()  
myStack
```

#Thực hành 1.2:

```
from collections import deque      #??  
  
myStack = deque()  
  
myStack.append('data science')     #??  
myStack.append('data structures and algorithms')  
myStack.append('learning data analytics')  
myStack.append('big data')  
myStack  
  
myStack.pop()                      #??  
myStack.pop()  
myStack
```

#Thực hành 1.3:

```
class Stack:
    def __init__(self):
        self.stack = []

    def add(self, data):
        #danh sách thêm phần tử data
        if data not in self.stack:
            self.stack.append(data)
            return True
        else:
            return False

    #hoạt động xoá phần tử data
    def remove(self):
        if len(self.stack) <= 0:
            return ("No element in the Stack")
        else:
            return self.stack.pop()

AStack = Stack()
AStack.add("data science")
AStack.add("big data")
AStack.add("IoT")
AStack.add("deep learning")
AStack.add("machine learning")
print(AStack.remove())
```

#Thực hành 1.4:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self):
        self.head = Node("head")
        self.size = 0

    def __str__(self):
        cur = self.head.next
        out = ""
        while cur:
            out += str(cur.value) + "->"
            cur = cur.next
        return out[:-3]

    def getSize(self):
        return self.size

    def isEmpty(self):
        return self.size == 0

    def peek(self):
        if self.isEmpty():
            raise Exception("Peeking stack")
        return self.head.next.value

    def push(self, value):
        node = Node(value)
        node.next = self.head.next
        self.head.next = node
        self.size += 1

    def pop(self):
        if self.isEmpty():
            raise Exception("Popping from an
empty stack")

        remove = self.head.next
        self.head.next = self.head.next.next
        self.size -= 1
        return remove.value

if __name__ == "__main__":
    stack = Stack()
    for i in range(1, 20):
        stack.push(i)
    print(f"Stack: {stack}")
    for _ in range(1, 9):
        remove = stack.pop()
        print(f"Pop: {remove}")
    print(f"Stack: {stack}")
```

#Thực hành 2.1:

myQueue = [] #??

myQueue.append('data science') #??

myQueue.append('data analytics')

myQueue.append('data structures and algorithms')

myQueue.append('big data')

myQueue.append('learning data analytics')

print(myQueue)

print(myQueue.pop(0)) #??

print(myQueue.pop(0))

print(myQueue)

#Thực hành 2.2:

```
from collections import deque

q = deque()                                #??

q.append('data analytics')                 #??

q.append('data structures and algorithms')

q.append('big data')

q.append('learning data analytics')

print(q)

print(q.popleft())                         #??

print(q.popleft())

print(q)
```

#Thực hành 2.3:

```
from queue import Queue
```

```
q = Queue(maxsize = 5)
```

```
print(q.qsize())
```

```
q.put('data analytics') #??
```

```
q.put('data structures and algorithms')
```

```
q.put('big data')
```

```
q.put('learning data analytics')
```

```
print(q.qsize())
```

```
print(q.get()) #??
```

```
print(q.get())
```

#Thực hành 2.4:

```
class Queue:
    def __init__(self):
        self.queue = list()

    def addque(self,data):
        #hoạt động chèn thêm phần tử data
        if data not in self.queue:
            self.queue.insert(0,data)
            return True
        return False

    def size(self):
        return len(self.queue)

TheQueue = Queue()
TheQueue.addque("data science")
TheQueue.addque("big data")
TheQueue.addque("IoT")
TheQueue.addque("machine learning")
TheQueue.addque("deep learning")
print(TheQueue.size())
```

